

5.4. Gradient Boosting Trees

You've been working hard, and now you have all the tools you need to build and tune models. We'll start this lesson the same way we've started the others: preparing the data and building our model, and this time with a new ensemble model. Once it's working, we'll learn some new performance metrics to evaluate it. By the end of this lesson, you'll have written your first Python module!

```
In [1]: import gzip
import json
import pickle

import ipywidgets as widgets
import pandas as pd
from imblearn.over_sampling import RandomOverSampler
from IPython.display import VimeoVideo
from ipywidgets import interact
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    classification_report,
    confusion_matrix,
)
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.pipeline import make_pipeline
from teaching_tools.widgets import ConfusionMatrixWidget
```

```
In [ ]: VimeoVideo("696221191", h="275ffd1421", width=600)
```

Prepare Data

All the data preparation for this module is the same as it was last time around. See you on the other side!

Import

Task 5.4.1: Complete the `wrangle` function below using the code you developed in the lesson 5.1. Then use it to import `poland-bankruptcy-data-2009.json.gz` into the DataFrame `df`.

- [Write a function in Python.](#)

```
In [5]: import pandas as pd
import gzip
```

```
import json

def wrangle(filename):
    # Open compressed file, load into dictionary
    with gzip.open(filename, "r") as f:
        data = json.load(f)

    # Load dictionary into DataFrame, set index
    df = pd.DataFrame().from_dict(data["data"]).set_index("company_id")

    return df

# Import the dataset using wrangle()
df = wrangle("data/poland-bankruptcy-data-2009.json.gz")

print(df.shape)
df.head()
```

(9977, 65)

```
Out[5]:
```

	feat_1	feat_2	feat_3	feat_4	feat_5	feat_6	feat_7	feat_8	feat_9
company_id									
1	0.174190	0.41299	0.14371	1.3480	-28.9820	0.60383	0.219460	1.12250	1.196
2	0.146240	0.46038	0.28230	1.6294	2.5952	0.00000	0.171850	1.17210	1.601
3	0.000595	0.22612	0.48839	3.1599	84.8740	0.19114	0.004572	2.98810	1.007
5	0.188290	0.41504	0.34231	1.9279	-58.2740	0.00000	0.233580	1.40940	1.339
6	0.182060	0.55615	0.32191	1.6045	16.3140	0.00000	0.182060	0.79808	1.812

5 rows × 65 columns



```
In [ ]: # df = ...
        # print(df.shape)
        # df.head()
```

Split

Task 5.4.2: Create your feature matrix `X` and target vector `y`. Your target is `"bankrupt"`.

- What's a feature matrix?
- What's a target vector?
- Subset a DataFrame by selecting one or more columns in pandas.
- Select a Series from a DataFrame in pandas.

```
In [6]: target = "bankrupt"

        # Feature matrix
        X = df.drop(columns=[target])
```

```
# Target vector
y = df[target]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (9977, 64)
y shape: (9977,)
```

```
In [ ]: # target = "bankrupt"
        # X = ...
        # y = ...

        # print("X shape:", X.shape)
        # print("y shape:", y.shape)
```

Task 5.4.3: Divide your data (`X` and `y`) into training and test sets using a randomized train-test split. Your test set should be 20% of your total data. And don't forget to set a `random_state` for reproducibility.

- Perform a randomized train-test split using scikit-learn.

```
In [ ]: # X_train, X_test, y_train, y_test = ...

        # print("X_train shape:", X_train.shape)
        # print("y_train shape:", y_train.shape)
        # print("X_test shape:", X_test.shape)
        # print("y_test shape:", y_test.shape)
```

```
In [7]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (7981, 64)
y_train shape: (7981,)
X_test shape: (1996, 64)
y_test shape: (1996,)
```

Resample

Task 5.4.4: Create a new feature matrix `X_train_over` and target vector `y_train_over` by performing random over-sampling on the training data.

- What is over-sampling?
- Perform random over-sampling using `imbalanced-learn`.

```
In [ ]: # over_sampler = ...
# X_train_over, y_train_over = ...
# print("X_train_over shape:", X_train_over.shape)
# X_train_over.head()
```

```
In [8]: from imblearn.over_sampling import RandomOverSampler

# Create the over sampler
over_sampler = RandomOverSampler(random_state=42)

# Apply random oversampling
X_train_over, y_train_over = over_sampler.fit_resample(X_train, y_train)

print("X_train_over shape:", X_train_over.shape)
X_train_over.head()
```

X_train_over shape: (15194, 64)

```
Out[8]:
```

	feat_1	feat_2	feat_3	feat_4	feat_5	feat_6	feat_7	feat_8	feat_9	feat_10
0	0.279320	0.053105	0.852030	17.0440	199.080	0.741770	0.353570	16.00600	1.2346	0.000000
1	0.001871	0.735120	0.156460	1.2269	-10.837	0.000000	0.002938	0.36032	1.4809	0.000000
2	0.113940	0.490250	0.077121	1.2332	-43.184	-0.000171	0.113940	1.03980	1.1649	0.000000
3	0.008136	0.652610	0.148120	1.2628	29.071	0.000000	0.008136	0.53230	1.2891	0.000000
4	0.045396	0.279640	0.708730	3.7656	238.120	0.000000	0.056710	2.57610	1.0169	0.000000

5 rows × 64 columns



Build Model

Now let's put together our model. We'll start by calculating the baseline accuracy, just like we did last time.

Baseline

Task 5.4.5: Calculate the baseline accuracy score for your model.

- What's accuracy score?
- Aggregate data in a Series using `value_counts` in pandas.

```
In [ ]: # acc_baseline = ...
        # print("Baseline Accuracy:", round(acc_baseline, 4))
```

```
In [9]: from sklearn.metrics import accuracy_score

        # Most common class in training target
        baseline_class = y_train.value_counts().idxmax()

        # Baseline predictions for test set
        y_pred_baseline = [baseline_class] * len(y_test)

        # Baseline accuracy
        acc_baseline = accuracy_score(y_test, y_pred_baseline)

        print("Baseline Accuracy:", round(acc_baseline, 4))
```

Baseline Accuracy: 0.9584

Iterate

Even though the building blocks are the same, here's where we start working with something new. First, we're going to use a new type of ensemble model for our classifier.

```
In [ ]: VimeoVideo("696221115", h="44fe95d5d9", width=600)
```

Task 5.4.6: Create a pipeline named `clf` (short for "classifier") that contains a `SimpleImputer` transformer and a `GradientBoostingClassifier` predictor.

- [What's an ensemble model?](#)
- [What's a gradient boosting model?](#)

```
In [10]: from sklearn.pipeline import make_pipeline
         from sklearn.impute import SimpleImputer
         from sklearn.ensemble import GradientBoostingClassifier

         clf = make_pipeline(
             SimpleImputer(),
             GradientBoostingClassifier(random_state=42)
         )
```

Remember while we're doing this that we only want to be looking at the *positive* class. Here, the positive class is the one where the companies really did go bankrupt. In the dictionary we made last time, the positive class is made up of the companies with the `bankrupt: true` key-value pair.

Next, we're going to tune some of the hyperparameters for our model.

```
In [ ]: VimeoVideo("696221055", h="b675d7fec0", width=600)
```

Task 5.4.7: Create a dictionary with the range of hyperparameters that we want to evaluate for our classifier.

1. For the `SimpleImputer`, try both the `"mean"` and `"median"` strategies.
2. For the `GradientBoostingClassifier`, try `max_depth` settings between 2 and 5.
3. Also for the `GradientBoostingClassifier`, try `n_estimators` settings between 20 and 31, by steps of 5.

- [What's a dictionary?](#)
- [What's a hyperparameter?](#)
- [Create a range in Python.](#)
- [Define a hyperparameter grid for model tuning in scikit-learn.](#)

```
In [ ]: # params = ...  
# params
```

```
In [11]: params = {  
    "simpleimputer__strategy": ["mean", "median"],  
    "gradientboostingclassifier__max_depth": [2, 3, 4, 5],  
    "gradientboostingclassifier__n_estimators": [20, 25, 30]  
}  
params
```

```
Out[11]: {'simpleimputer__strategy': ['mean', 'median'],  
          'gradientboostingclassifier__max_depth': [2, 3, 4, 5],  
          'gradientboostingclassifier__n_estimators': [20, 25, 30]}
```

Note that we're trying much smaller numbers of `n_estimators`. This is because `GradientBoostingClassifier` is slower to train than the `RandomForestClassifier`. You can try increasing the number of estimators to see if model performance improves, but keep in mind that you could be waiting a long time!

```
In [ ]: VimeoVideo("696221023", h="218915d38e", width=600)
```

Task 5.4.8: Create a `GridSearchCV` named `model` that includes your classifier and hyperparameter grid. Be sure to use the same arguments for `cv` and `n_jobs` that you used above, and set `verbose` to 1.

- [What's cross-validation?](#)
- [What's a grid search?](#)
- [Perform a hyperparameter grid search in scikit-learn.](#)

```
In [12]: from sklearn.model_selection import GridSearchCV  
  
model = GridSearchCV(  
    estimator=clf,  
    param_grid=params,  
    cv=5,  
    n_jobs=-1,
```

```
verbose=1  
)
```

Now that we have everything we need for the model, let's fit it to the data and see what we've got.

```
In [ ]: VimeoVideo("696220978", h="008d915f33", width=600)
```

Task 5.4.9: Fit your `model` to the over-sampled training data.

```
In [ ]: # Fit model to over-sampled training data  
model.fit(X_train_over, y_train_over)
```

Fitting 5 folds for each of 24 candidates, totalling 120 fits

This will take longer than our last grid search, so now's a good time to get coffee or cook dinner. ☺

Okay! Let's take a look at the results!

```
In [ ]: VimeoVideo("696220937", h="9148032400", width=600)
```

Task 5.4.10: Extract the cross-validation results from `model` and load them into a DataFrame named `results`.

- Get cross-validation results from a hyperparameter search in scikit-learn.

```
In [ ]: # results = ...  
# results.sort_values("rank_test_score").head(10)
```

```
In [ ]: import pandas as pd  
  
results = pd.DataFrame(model.cv_results_)  
results.sort_values("rank_test_score").head(10)
```

There are quite a few hyperparameters there, so let's pull out the ones that work best for our model.

```
In [ ]: VimeoVideo("696220899", h="342d55e7d7", width=600)
```

Task 5.4.11: Extract the best hyperparameters from `model`.

- Get settings from a hyperparameter search in scikit-learn.

```
In [ ]: # Extract best hyperparameters  
best_params = model.best_params_  
best_params
```

Evaluate

Now that we have a working model that's actually giving us something useful, let's see how good it really is.

Task 5.4.12: Calculate the training and test accuracy scores for `model` .

- [Calculate the accuracy score for a model in scikit-learn.](#)

```
In [ ]: acc_train = model.score(X_train_over, y_train_over)
        acc_test = model.score(X_test, y_test)

        print("Training Accuracy:", round(acc_train, 4))
        print("Validation Accuracy:", round(acc_test, 4))
```

```
In [ ]: # acc_train = ...
        # acc_test = ...

        # print("Training Accuracy:", round(acc_train, 4))
        # print("Validation Accuracy:", round(acc_test, 4))
```

Just like before, let's make a confusion matrix to see how our model is making its correct and incorrect predictions.

Task 5.4.13: Plot a confusion matrix that shows how your best model performs on your test set.

- [What's a confusion matrix?](#)
- [Create a confusion matrix using scikit-learn.](#)

```
In [ ]: # Plot confusion matrix
        from sklearn.metrics import ConfusionMatrixDisplay
        import matplotlib.pyplot as plt

        # Plot confusion matrix using the best estimator
        ConfusionMatrixDisplay.from_estimator(
            model.best_estimator_,
            X_test,
            y_test,
            cmap="Blues"
        )

        plt.title("Confusion Matrix")
        plt.show()
```

This matrix is a great reminder of how imbalanced our data is, and of why accuracy isn't always the best metric for judging whether or not a model is giving us what we want. After all, if 95% of the companies in our dataset didn't go bankrupt, all the model has to do is always predict `{"bankrupt": False}` , and it'll be right 95% of the time. The accuracy score will be amazing, but it won't tell us what we really need to know.

Instead, we can evaluate our model using two new metrics: **precision** and **recall**. The precision score is important when we want our model to only predict that a company will go bankrupt if its very confident in its prediction. The *recall* score is important if we want to make sure to identify all the companies that will go bankrupt, even if that means being incorrect sometimes.

Let's start with a report you can create with scikit-learn to calculate both metrics. Then we'll look at them one-by-one using a visualization tool we've built especially for the Data Science Lab.

```
In [ ]: VimeoVideo("696297886", h="fac5454b22", width=600)
```

Task 5.4.14: Print the classification report for your model, using the test set.

- [Generate a classification report with scikit-learn.](#)

```
In [ ]: # Print classification report
# print(...)
```

```
In [ ]: from sklearn.metrics import classification_report

y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
```

```
In [ ]: VimeoVideo("696220837", h="f93be5aba0", width=600)
```

```
In [ ]: VimeoVideo("696220785", h="8a4c4bfff58", width=600)
```

Task 5.4.15: Run the cell below to load the confusion matrix widget.

- [What's precision?](#)
- [What's recall?](#)

```
In [ ]: c = ConfusionMatrixWidget(model, X_test, y_test)
c.show()
```

If you move the probability threshold, you can see that there's a tradeoff between precision and recall. That is, as one gets better, the other suffers. As a data scientist, you'll often need to decide whether you want a model with better precision or better recall. What you choose will depend on how to intend to use your model.

Let's look at two examples, one where recall is the priority and one where precision is more important. First, let's say you work for a regulatory agency in the European Union that assists companies and investors navigate [insolvency proceedings](#). You want to build a model to predict which companies could go bankrupt so that you can send debtors information about filing for legal protection before their company becomes insolvent. The administrative costs

of sending information to a company is €500. The legal costs to the European court system if a company doesn't file for protection before bankruptcy is €50,000.

For a model like this, we want to focus on **recall**, because recall is all about *quantity*. A model that prioritizes recall will cast the widest possible net, which is the way to approach this problem. We want to send information to as many potentially-bankrupt companies as possible, because it costs a lot less to send information to a company that might not become insolvent than it does to skip a company that does.

```
In [ ]: VimeoVideo("696209314", h="36a14b503c", width=600)
```

Task 5.4.16: Run the cell below, and use the slider to change the probability threshold of your model. What relationship do you see between changes in the threshold and changes in wasted administrative and legal costs? In your opinion, which is more important for this model: high precision or high recall?

- [What's precision?](#)
- [What's recall?](#)

```
In [ ]: c.show_eu()
```

For the second example, let's say we work at a private equity firm that purchases distressed businesses, improve them, and then sells them for a profit. You want to build a model to predict which companies will go bankrupt so that you can purchase them ahead of your competitors. If the firm purchases a company that is indeed insolvent, it can make a profit of €100 million or more. But if it purchases a company that isn't insolvent and can't be resold at a profit, the firm will lose €250 million.

For a model like this, we want to focus on **precision**. If we're trying to maximize our profit, the *quality* of our predictions is much more important than the *quantity* of our predictions. It's not a big deal if we don't catch every single insolvent company, but it's *definitely* a big deal if the companies we catch don't end up becoming insolvent.

This time we're going to build the visualization together.

```
In [ ]: VimeoVideo("696209348", h="f7e1981c9f", width=600)
```

Task 5.4.17: Create an interactive dashboard that shows how company profit and losses change in relationship to your model's probability threshold. Start with the `make_cnf_matrix` function, which should calculate and print profit/losses, and display a confusion matrix. Then create a FloatSlider `thresh_widget` that ranges from 0 to 1. Finally combine your function and slider in the `interact` function.

- [What's a function?](#)
- [What's a confusion matrix?](#)
- [Create a confusion matrix using scikit-learn.](#)

```
In [ ]: def make_cnf_matrix(threshold):  
        pass  
  
        thresh_widget = ...  
  
        interact(make_cnf_matrix, threshold=thresh_widget);
```

Go Further:💡 Some students have suggested that this widget would be better if it showed the sum of profits and losses. Can you add that total?

Communicate

Almost there! Save the best model so we can share it with other people, then put it all together with what we learned in the last lesson.

Task 5.4.18: Using a context manager, save your best-performing model to a file named "model-5-4.pkl".

- [What's serialization?](#)
- [Store a Python object as a serialized file using pickle.](#)

```
In [ ]: # Save model
```

```
In [ ]: VimeoVideo("696220731", h="8086ff0bcd", width=600)
```

Task 5.4.19: Open the file `my_predictor_lesson.py`, add the `wrangle` and `make_predictions` functions from the last lesson, and add all the necessary import statements to the top of the file. Once you're done, save the file. You can check that the contents are correct by running the cell below.

- [What's a function?](#)

```
In [ ]: %%bash  
  
cat my_predictor_lesson.py
```

Congratulations: You've created your first module!

```
In [ ]: VimeoVideo("696220643", h="8a3f141262", width=600)
```

Task 5.4.20: Import your `make_predictions` function from your `my_predictor` module, and use the code below to make sure it works as expected. Once you're satisfied, submit it to the grader.

```
In [ ]: # Import your module

# Generate predictions
y_test_pred = make_predictions(
    data_filepath="data/poland-bankruptcy-data-2009-mvp-features.json.gz",
    model_filepath="model-5-4.pkl",
)

print("predictions shape:", y_test_pred.shape)
y_test_pred.head()
```

Copyright 2023 WorldQuant University. This content is licensed solely for personal use.
Redistribution or publication of this material is strictly prohibited.